

CSC 330 Programming Languages

Lecture #4

Type Systems

There are far more to data types than just deciding what a variable should be. Different languages have different type systems and there are a lot of design decisions to be made here.

What is a Data Type?

First off, what is a type? You would probably answer this by stating different data types that you know, such as int, float, string, etc. But stating an example isn't explaining the concept. So, what is a data type? All data is stored as binary. This means that to the computer, your variables really look like this:

```
var1 = 10101100
var2 = 01100100
var3 = 00000001
```

Now of course we can name our variables appropriately to give the human reader some context, but this does nothing to help the computer:

```
count = 10101100
letter = 01100100
isAttending = 00000001
```

The values are still binary, and the computer must know how to interpret the bits. Are these numbers, booleans, letters, pointers to memory locations? The only way that the computer can answer that question is to look at the data type of each variable:

```
int count = 10101100 // interpreted as the int value 172
char letter = 01100100 // interpreted as the letter 'd' (using ASCII)
boolean isAttending = 00000001 // interpreted as the boolean value true
```

So, a data type can be thought of as an interpretation of bits. Actually, this may shed some light on what casting is:

```
value = 00000000

// now 00000000 is interpreted as the number zero
int num = (int)value;

// now 00000000 is interpreted as false
bool b = (bool)value;

// now 00000000 is interpreted as the null termination character '\0'
char ch = (char)value;

// now 00000000 is interpreted as a pointer to a memory location
// which is memory location zero, aka "null"
const char* s = (const char*)value;
```

In most cases, a cast simply applies a new interpretation of the bits representing a value. In fact, this data type definition applies to classes and structs as well. Consider the following value:

```
value = 010101000000000001000011011000010111010000000000
```

Again, the data type specifies how to interpret these bits. Well, what is the data type of the variable "value"? Let's say the data type is "MyType" which is specified as the following:

```
class MyType {  
    byte number;  
    boolean flag;  
    String msg;  
}  
MyType value;
```

Even if we have methods to our class, the binary representation only encodes the fields, in the exact order that they appear. To see this better, let's look at the bits behind value again:

```
value = 01010100 00000000 01000011011000010111010000000000  
        number    flag      msg
```

This means that if you pull each field out, you will get the following:

```
value.number is 01010100 // which is 84 since number is an int  
value.flag   is 00000000 // which is false since flag is a boolean  
value.msg     is 01000011 01100001 01110100 00000000  
// which is   'C'      'a'      't'      '\0' (signifies end of string)
```

Which further shows you that classes are really just data types. In fact classes, structs, interfaces, enums, records, etc are all just data types, which is just a way of interpreting bits.

Hungarian Notation

Some people add prefixes to variable names to indicate their data type. For example, adding lowercase i for integers, or s for strings. This is known as Hungarian notation. Here are some examples:

- iCount
- fHeight
- sName
- bFlag

This comes in handy when working with GUIs. There are several prefixes that can be applicable here:

- lbl for labels
- txt for textboxes
- chk for checkboxes
- rad for radio buttons
- img for images

Even putting an initial underscore on private variables can be a form of Hungarian notation. While this isn't widely used by all developers, it can be beneficial and can lead to more consistent habits.¹

¹ <https://www.joelonsoftware.com/2005/05/11/making-wrong-code-look-wrong/>

What is a Type System?

By now you should understand binary arithmetic. We can add bit strings like so:

```
  01101110
+ 00011001
=====
  10000111
```

But does the answer make sense? If these are integers, then we are essentially encoding the logic of $110 + 25 = 135$, which is logical and correct. However, if these are memory locations, then numerically adding them doesn't seem too helpful. If these were characters or strings, adding them numerically also wouldn't make much sense. So the question becomes, is this a meaningful and correct operation for the data types. We should only allow this if it makes sense. So even though we CAN add the bit strings, does it make SENSE to? If it doesn't, the programming language should complain. Otherwise, it should allow the operation to be performed. This is part of the job of the type system.

For this to work, all variables should have a data type. If they do, then we always know how to interpret their current value and decide whether a given operation is valid for the type. There are various type systems and different languages will choose different approaches to take for their type system. Let's look at some of the differences in type systems.

Static v. Dynamic

A static type system binds its data types during compilation. This means that the compiler must be ok with the data types and operations used on them. For example, the following will not compile:

```
int num = 3;
boolean flag = true;
System.out.println(num + flag);
```

The compiler knows the type of num is an int and the type of flag is a boolean. Without having to actually run the code, the compiler can figure out that num plus flag is not a supported operation. There are other things we have to worry about in a static type system, such as operations on a class. For example:

```
class Fruit {
    public void eat() {
    }
}
Fruit f = new Fruit();
f.consume();
```

This will also not compile. The compiler can see that f is of type Fruit. It knows all the methods available to Fruit and it knows that "consume" is not one of them. While languages such as Python will allow this and will result in a failure at runtime, static type systems will check these things at compile time meaning we don't even make it to execution before the error is caught. This can be a blessing as many errors can be caught ahead of time, but it can also be frustrating. For example:

```

class Fruit {
    public void eat() {
    }
}
class Apple extends Fruit {
    public void consume() {
    }
}
Fruit f = new Apple();
f.consume();

```

Now will the code above compile? We added a new class and set the variable f to be an object of that class. So, we should be good right? Try this code out in Java and tell me what you get. I'll wait... . Were you surprised to see an error? This code, if allowed to run, would actually execute properly. But the compiler doesn't know that. This is because the compiler only knows about the "compile time type". This is the datatype that the variable is declared as. For f this is Fruit. There is also something known as the "runtime type". This is the datatype the variable actually points to. This is only known at runtime and is not something the compiler uses, aside from the initial check to make sure that the runtime type is valid according to the compile time type. For example, if the compile time type is Fruit, then the runtime type must either be Fruit or a subclass of Fruit. Past that, it really doesn't use that information. You may think this is really dumb of the compiler, but imagine this situation:

```

Fruit f;
System.out.println("Enter either 0 or 1: ");
int choice = new Scanner(System.in).nextInt();
if (choice == 0) {
    f = new Apple();
}
else if (choice == 1) {
    f = new Fruit();
}
f.consume();

```

How about now? Is the compiler still being dumb? In this case we have a 50/50 shot of f being an Apple or a Fruit (as long as the user follows directions). In this case, the call to consume() might fail, or it might not. There are no guarantees. For this reason, the compiler will not use the runtime type to determine if an operation is safe or not. It will only use the compile time type. This means that if we want this code to compile, the consume method must be put into the Fruit class. Of course, the Apple class can override it:

```

class Fruit {
    public void eat() { }
    public void consume() { }
}
class Apple extends Fruit {
    @Override
    public void consume() { }
}

```

Now the compiler is perfectly happy. It knows that no matter what happens, the method consume will exist and will be called. The exact version of consume is not something the compiler knows but it knows that some version of consume will be called. And its ok with that. If you want to learn more about how the runtime

decides which version of consume to call, look up "virtual function tables". This topic is very interesting but is outside the scope of this course.

How about if we have two similar classes. For example:

```
class Dog {
    public void speak() { }
}

class Cat {
    public void speak() { }
}

Dog d = null;
Cat c = null;
System.out.print("Choose either d for dog or c for cat: ");
char choice = new Scanner(System.in).next().charAt(0);
if (choice == 'c') {
    c = new Cat();
}
else if (choice == 'd') {
    d = new Dog();
}
// which one got created?
// which one do we tell to speak?
```

In this situation we have two variables. Only one of them is actually initialized. However, outside of the if statement, we have no idea which one was created. If we tell the dog to speak yet the cat was instantiated (or vice versa) we will get a `NullPointerException`. So only one can speak yet which one is it? Perhaps we could create a single variable for an animal and just use that. In a dynamic system like what Python has, this would be just fine:

```
choice = input('Choose either d for dog or c for cat: ')
if (choice == 'c'):
    animal = Cat()
elif (choice == 'd'):
    animal = Dog()
animal.speak()
```

But we can't do this in a static type system. Remember that the compiler must be sure that the `speak` method exists. It has to know this without executing the code. We also must create the variable "animal" before using it, and we can only give it one data type. Due to these restrictions, we will be forced to create what is called a "marker class":

```
class Animal { }

class Cat extends Animal {
    public void speak() { }
}
```

```
class Dog extends Animal {
    public void speak() { }
}
```

Now we can alter the code:

```
Animal animal = null;
System.out.print("Choose either d for dog or c for cat: ");
char choice = new Scanner(System.in).next().charAt(0);
if (choice == 'c') {
    animal = new Cat();
}
else if (choice == 'd') {
    animal = new Dog();
}
animal.speak();
```

Not so fast! The speak method is not a part of this marker class. Remember we only use the compile time type to check if a method exists to be called. So, speak must be a part of the Animal class.

```
class Animal {
    public void speak() { }
}
```

Again, the subclasses can override speak if need be, but now the compiler is happy. There was nothing technically wrong with the code and it would have executed just fine. But sometimes we have to do things to make the static type system happy.

Static type systems are great because they help catch a bunch of potential errors. It can check your entire code base to make sure there aren't any type issues. While type issues are still possible at runtime, a static type system will catch the majority of them. But for that convenience, we have to spend more time making the type system happy. So, it's a tradeoff.

A dynamic type system will happily wait until execution to figure out if types match up. This means less time trying to just get your code to compile, but more time tracking down bugs at execution. It also means that you might have a type error in some random spot in the code that is rarely executed. For example:

```
choice = input("Enter any number: ")
if (choice == 42):
    print("hello" + 14)
```

We may run this code hundreds of times with many different users and it never crashes. Until that one fateful day when someone enters in 42. Remember that in a dynamic type system, the runtime checks types. If the runtime doesn't actually run the code, it doesn't know there is a type mismatch. So this code might be executed many times over many days/weeks/months without ever realizing there is a landmine right there in that if statement.

There isn't a clear winner here. Some people like static, some like dynamic, some like both depending on the situation and the project they are working on. They both have their pros and cons.

Manifest v. Implicit

You may get the impression looking at Java and Python that a static type system is one in which you specify the data type and a dynamic type system is one in which you don't. This is actually not true. Take the language ML for example. Here is a small code snippet from the language:

```
(* ML *)  
val a = 3;  
val b = "hello";  
print a + b;
```

This code will not compile. ML uses a static type system but types do not have to be specified. The compiler was able to figure out that a is an int and b is a string. It used "type deduction" (also known as "type inference") which basically means that it evaluated what a was being assigned to, figured out its type and used that type for a. In that way the compiler can figure out a's type and use that information. This is known as an "implicit static type system". The word "implicit" means that we don't have to specify a data type.

This is different from a "manifest static type system". In this system, the data type must be specified. For example:

```
// Java  
int a = 3;  
String b = "hello";
```

So having a static type system does NOT mean that you must specify the data type. Having a manifest static type system does, but you could also have an implicit static type system in which case you don't have to. It is interesting to note that although Java, C# and C++ all use manifest type systems, they do have a keyword that still gives this ability:

```
// in Java and C#  
var a = 3;  
  
// in C++  
auto a = 3;
```

The "var" (in Java and C#) or "auto" (in C++) keyword tells the compiler to deduce the data type from the assignment.

To be fair, Python does have a way to specify types. I wouldn't necessarily call Python's type system a manifest dynamic type system, but the capabilities are there with "type hints":

```
# Python  
a : int = 3;  
b : str = "hello";
```

This feature gives developers the ability to be more clear about data types. However, Python's type system mostly ignores this, as the runtime still deduces the types, so this is still an implicit type system. Type hints are mostly for human developers reading your code. Of course, there are tools (such as "mypy") that will use the type hints and tell you if any types are being misused.

Strong v. Weak

You might think that a static type system is "stronger" than a dynamic type system. This is a common misconception and a misunderstanding of terminology. The terms "strong type system" and "weak type system" actually have nothing to do with whether a type system is static or dynamic. All strong v. weak means is whether the type system is more forgiving when it comes to mixing up data types. For example, is the following ok?

```
"hello" + 3
```

You might automatically say "no". Not so fast on that. In Python, you would be correct as this is NOT ok. Try it out. However, in Java this is perfectly fine. In Java the 3 will be turned into a string (i.e. "3") and then concatenated to "hello" to produce "hello3". Python has a dynamic type system, yet it is said to be "stronger" than Java. In other words, Python doesn't let you get away with mixing types as much as Java does. C/C++ is worse:

```
// C
#include <stdio.h>
#include <stdlib.h>

int main() {
    int* ptr = (int*)"hello";
    printf("%d", *ptr);
}
```

Want to know what casting "hello" to an integer produces? Well, compile and run the above C code. C doesn't complain at all about this. In general, casting is largely unchecked in the C and C++ language so invalid casts are not caught by the compiler. This leads C and C++ to have a pretty weak type system.

Javascript has probably the weakest type system I have ever seen. It pretty much let's you combine types in any way.

```
// Javascript
[] + []      // empty array plus empty array gives an empty string
[] * []      // empty array times empty array gives zero
[] + {}      // empty array plus empty object gives a string '[object Object]'
{} + []      // however, empty object plus empty array gives zero
0 + null     // this is also somehow zero
null + 0     // also zero. At least this is consistent
null + 5     // gives 5, which is consistent if we think of null as zero.
null + "5"   // gives the string "null5". I have no words for this.
```

There is actually a funny video on this². ML has probably the strongest type system. All types must match. Even something like this is illegal:

```
(* ML *)
val a = 3;
val b = 3.0;
print a + b;
```

This won't compile since an int and a float are not the same type. You can only add two ints or two floats.

² <https://www.destroyallsoftware.com/talks/wat>

Widening v Narrowing Conversions

C# and some other languages have an interesting way of dealing with these type mismatches. C# will allow for an implicit type conversion on numbers but only if it is a "widening" conversion. This means the type we convert into must have a larger range of values or precision. For example there isn't any harm in converting a 16 bit integer into a 32 bit integer. Having more bits just means we can store more numbers. However, going from a 32 bit integer into a 16 bit is known as a "narrowing" conversion. In this case data could be lost or altered. Any narrowing conversion must have an explicit cast. Here are some examples:

```
// C#
byte b = 5;
b + 300; // will automatically cast b to be an int since 300 is an int
int i = 50;
byte b = i; // this won't compile. Converting int to byte may alter data,
            // even if the current value can fit
byte b = (byte) i; // so an explicit cast is required

void foo(float f); // given this function
foo(5); // the int 5 will automatically be converted into a float,
        // since a float is a wider type than int
double d = 6.0;
foo(d); // this won't compile since float is a narrower type than a
        // double, so an implicit cast won't be made
foo((float)d); // this is required for compilation to succeed
foo(6.0); // oddly enough this won't compile. Fractional numbers default
          // to double, so this is actually a narrowing conversion
foo(6.0f); // this is fine since appending an "f" (or "F") to the number
           // explicitly casts it to a float
```

More information can be found online³. Languages like Java also tend to follow similar rules where widening conversions are implicit and narrowing conversion require explicit casts.

Best Candidate Algorithm

Given the following C# code:

```
// C#
public class Program {
    public static void Main(string[] args) {
        System.Console.WriteLine(foo(5));
    }
    private static int foo(float f) { return 2; }
}
```

What will the output be? Well considering how we just went over widening conversions; you should be able to guess that the integer 5 will be implicitly converted into a float so the function foo can be called. This results in the number 2 being displayed.

³ <https://learn.microsoft.com/en-us/dotnet/standard/base-types/conversion-tables>

How about now?

```
// C#
public class Program {
    public static void Main(string[] args) {
        System.Console.WriteLine(foo(5));
    }
    private static int foo(int i) { return 1; }
    private static int foo(float f) { return 2; }
}
```

There seems to be a better match. Remember that languages like C#, Java and C++ support functions with the same name. This is known as "function overloading" (not to be confused with "operator overloading" or "function overriding" which are both separate concepts). As long as the parameters are different, this is allowed. Note that languages like C and Python do NOT allow this (I'm sure they have their reasons). In C this will result in a compiler error and in Python the function defined later will shadow the function defined earlier, making it impossible to call the earlier function.

Function overloading is a great feature, but it relies upon an algorithm, which is sometimes called the "best candidate" algorithm. The idea is to look at the argument(s) being passed and choose the function that has parameter(s) with the best match in data type(s). This can sometimes be a straightforward process. In the code example above, clearly `foo(int i)` is a better match since no conversion is needed for the argument 5. So now the code produces the number 1. How about this:

```
// C#
public class Program {
    public static void Main(string[] args) {
        System.Console.WriteLine(foo(1, 1.0f));
        System.Console.WriteLine(foo(1.0f, 1));
        System.Console.WriteLine(foo(1, 1));
        System.Console.WriteLine(foo(1.0f, 1.0f));
    }

    private static int foo(int i, float f) {
        return 1;
    }
    private static int foo(float f, int i) {
        return 2;
    }
}
```

It should be clear that the first call will use `foo(int, float)` and the second call will use `foo(float, int)`. What about the third? We know C# will implicitly convert an int into a float, but which one will it convert? Well, in this case you will get an error:

```
Main.cs(23,34): error CS0121: The call is ambiguous between the following methods or properties: 'Program.foo(int, float)' and 'Program.foo(float, int)'
```

The algorithm considers both functions to be at the same level of appropriateness so it can't decide.

How about the fourth call? That one produces this error:

```
Main.cs(24,38): error CS1503: Argument 1: cannot convert from 'float' to 'int'
```

Remember a float cannot be implicitly converted into an int. We would have to explicitly convert one of the arguments. Whichever one we convert will determine the function that gets called.

I think the C++ version of this code gives better error messages:

```
// C++
#include <iostream>
using namespace std;

int foo(int i, float f) {
    return 1;
}
int foo(float f, int i) {
    return 2;
}
int main() {
    cout << foo(1, 1.0f);
    cout << foo(1.0f, 1);
    cout << foo(1, 1);
    cout << foo(1.0f, 1.0f);
}
```

When attempting to compile, we get the following:

```
Main.cpp: In function 'int main()':
```

```
Main.cpp:13:21: error: call of overloaded 'foo(int, int)' is ambiguous
    cout << foo(1, 1);
                   ^
```

```
Main.cpp:4:5: note: candidate: int foo(int, float)
    int foo(int i, float f) {
    ^~~
```

```
Main.cpp:7:5: note: candidate: int foo(float, int)
    int foo(float f, int i) {
    ^~~
```

```
Main.cpp:14:27: error: call of overloaded 'foo(float, float)' is ambiguous
    cout << foo(1.0f, 1.0f);
                   ^
```

```
Main.cpp:4:5: note: candidate: int foo(int, float)
    int foo(int i, float f) {
    ^~~
```

```
Main.cpp:7:5: note: candidate: int foo(float, int)
    int foo(float f, int i) {
    ^~~
```

This best candidate algorithm must consider classes as well. Look at this C# code:

```
// C#
class Fruit {
}
class Apple : Fruit {
}

public class Program {
    public static void Main(string[] args) {
        System.Console.WriteLine(foo(new Fruit()));
        System.Console.WriteLine(foo(new Apple()));
    }
    private static int foo(Fruit f) { return 1; }
    private static int foo(Apple a) { return 2; }
}
```

Clearly the code will produce 1 followed by 2 in the terminal (on different lines).

Now let's try something trickier. Consider this:

```
// C#
class Fruit {
}
class Apple : Fruit {
}

public class Program {
    public static void Main(string[] args) {
        Fruit f = new Apple();
        System.Console.WriteLine(foo(f));
    }
    private static int foo(Fruit f) {
        return 1;
    }
    private static int foo(Apple a) {
        return 2;
    }
}
```

Now what gets printed? This is not nearly as straight forward, and you would be forgiven for not knowing the answer. On the one hand, the compile time type of `f` is `Fruit`, so `foo(Fruit)` should be called. But on the other hand the runtime type of `f` is `Apple`, so `foo(Apple)` should be called.

You may say that the compiler only looks at the compile time type, but consider the following code:

```
// C#
class Fruit {
    public virtual int foo() {
        return 1;
    }
}
class Apple : Fruit {
    public override int foo() {
        return 2;
    }
}

public class Program {
    public static void Main(string[] args) {
        Fruit f = new Apple();
        System.Console.WriteLine(f.foo());
    }
}
```

In this example we are using function overriding (not function overloading). Here the compiler still only sees the data type of `Fruit` for `f`, but we know that it will call `Apple`'s `foo`. This is just basic polymorphism, so you should be familiar with what is going on here. So, will the same thing happen with function overloading?

As it turns out, no! The ability of a language to determine which function to call based on either compile time type or runtime type is known as "dispatch". The concept of "single dispatch" means that the object the function is called upon uses its runtime time to disambiguate the function to be called. In other words, with `f.foo()` we look at the runtime type of `f` to determine which `foo` to call. This is accomplished using virtual function tables, which was briefly mentioned earlier (again Google to find out more). This is the first level of dispatch. The second level involves the arguments. In "double dispatch" (or "multiple dispatch") we look at the runtime type of the arguments to determine which function to call. So for `foo(f)` we would call `foo(Apple)`. HOWEVER, not all languages support double dispatch. In fact, most popular ones don't! C#, C++ and Java all support single dispatch only. So for these languages, we would actually call `foo(Fruit)`. Told you it wasn't an easy question!

It actually gets more interesting. C# 4.0 introduced the keyword "dynamic". This is used in place of a data type and causes the variable to delay its type binding until runtime. This means the variable behaves more like a dynamically typed language. For example:

```
// C#
dynamic counter = 3;
```

The compiler will ignore any type checks on this variable and delay those type checks until runtime. So doing things like this will actually compile:

```
// C#
System.Console.Write(counter + true);
```

Of course, the program will crash with a type error as soon as it is executed, but the compiler will let it through. Because it delays type decisions until runtime, we can actually mimic double dispatch in C# by casting the variable to a dynamic type:

```
// C#
class Fruit {
}
class Apple : Fruit {
}

public class Program {
    public static void Main(string[] args) {
        Fruit f = new Apple();
        System.Console.WriteLine(foo((dynamic)f));
    }
    private static int foo(Fruit f) {
        return 1;
    }
    private static int foo(Apple a) {
        return 2;
    }
}
```

Even though the compile time type is `Fruit`, the cast to `dynamic` tells the compiler to not run the best candidate algorithm, delaying that decision until runtime. Runtime will obviously use the runtime type and so it will conclude that `foo(Apple)` is the most appropriate and that function will be called.

Generics

You should already know about generics, at least when it comes to Java. Recall that generics are a way for a static type system to allow for flexibility with its types. You can add generics to a class or just to an individual method. Here is a simple program using generics in Java:

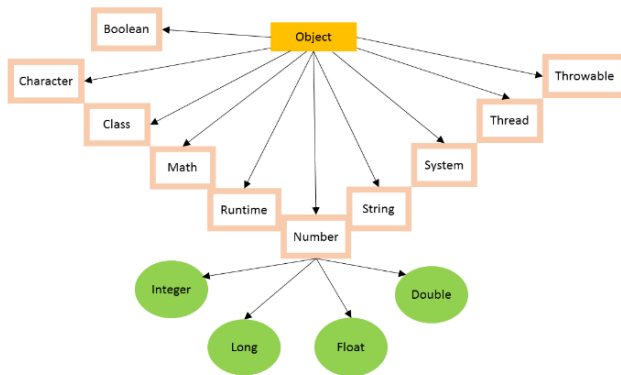
```
// Java
class Generics<T> {
    public T data;

    public Generics(T data) {
        this.data = data;
    }
}

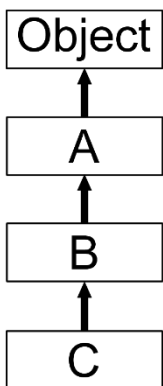
public class Main {
    public static void main(String[] args) {
        Generics<Integer> obj = new Generics<>(42);
        System.out.println(obj.data);
    }
}
```

A lot of statically typed languages have generics, however the way generics actually work behind the scenes is different for different languages. With Java, generics are just syntactic sugar. The compiler removes all generic

information during compilation. This is a process known as "type erasure". To understand type erasure, we need to cover something called a "unified type system". This means that all data types come from a single type. In Java, this type is called "Object". Java's type hierarchy looks something like this:



There are plenty of data types missing here but the point is that all data types inherit from Object in Java. This means that a String is an Object. An ArrayList, Queue, Stack, HashMap, etc. are an Object. Even classes that you create inherit from Object. For example, if you create a class called C that inherits from B, and B inherits from A, if A doesn't explicitly inherit from anything, Java automatically makes it inherit from Object:



Java only applies this system to classes, so primitive types like int, float, char, and boolean aren't included in this.

Since everything inherits from the Object class, Java can change any generic type into "Object". This will allow it to hold any subclass type. During type erasure, the compiler changes generic types into Object and then adds casts that will convert the Object into the generic type. This means you get type safety that is checked at compile time, but the runtime only sees casts and Object. This is also why you can't have a primitive type for the generic type parameter. For example:

Instead of this:

```
Generics<int> obj;  
Generics<float> obj;  
Generics<char> obj;  
Generics<boolean> obj;
```

You must use this:

```
Generics<Integer> obj;  
Generics<Float> obj;  
Generics<Character> obj;  
Generics<Boolean> obj;
```

C# has generics as well. The code looks similar to Java:

```
// C#
public class Generics<T> {
    public T data;

    public Generics(T data) {
        this.data = data;
    }
}

public class Program {
    public static void Main(string[] args) {
        Generics<int> obj = new(42);
        System.Console.WriteLine(obj.data);
    }
}
```

However, it works completely different. C# generics go all the way to the runtime and are not erased by the compiler. This means that generic information is retained and variables stay as their generic types. We can use primitive data types such as int, float, char, and bool as generic types. C# also has a unified type system but, unlike Java, its primitive types also inherit from the Object class. So, a primitive int is an Object.

Here is how C++ does it:

```
// C++
template <typename T>
class Generics {
public:
    T data;

    Generics(T data) {
        this->data = data;
    }
};

#include <iostream>
using namespace std;

int main() {
    Generics<int> obj(42);
    cout << obj.data << endl;
}
```

C++ uses something called "templates". Essentially, they are the same as generics. The way C++ handles generics is kind of a cross between Java and C#. The generics are erased during compilation, but they are replaced by several copies of the class with the generic replaced with a data type.

For example, if we had the following objects:

```
// C++
Generics<int> obj1(42);
Generics<float> obj2(42.5f);
```

C++ would take the Generics class and create two classes from it:

<u>int version</u>	<u>float version</u>
<pre>template <> class Generics<int> { public: int data; Generics(int data) { this->data = data; } };</pre>	<pre>template <> class Generics<float> { public: float data; Generics(float data) { this->data = data; } };</pre>

This process is known as "template instantiation". Generics are gone, but instead of casts we have multiple copies of the same class. This is important since C++ does NOT have a unified type system, so there is no "Object" class that we can set all the variables to. The compiler looks through the code and creates these class copies for us, but we could do it ourselves. If we wanted the int version of this class to be different, we could write our own template instantiation for it:

```
// C++
// template class
template <typename T>
class Generics {
public:
    T data;

    Generics(T data) {
        this->data = data;
    }
};

// if template type is int, use this one
template <>
class Generics<int> {
public:
    int data;

    Generics(int data) {
        this->data = data + 1;
    }
};
```

```

int main() {
    Generics<int> obj1(42);    // uses our int version
    Generics<float> obj2(42.5f); // compiler creates float version
    cout << obj1.data << endl; // displays 43
    cout << obj2.data << endl; // displays 42.5
}

```

Creating our own instantiations of the template class is known as "template specialization". The compiler will use our version if we created one (template specialization), otherwise it will create one (template instantiation) based on the template class. This is a pretty powerful feature!

Generics with Type Constraints

What if we wanted to apply restrictions or constraints to our generic type? For example, let's say that we wanted to set the generic field to null in the constructor. In Java, this would be ok since all generic types must be reference types (and all reference types are "nullable", i.e. can be set to null). However, in C++ and C#, this is a problem. A primitive type like int and float cannot be set to null. They are not nullable types. To see what I mean, try the following in C++ (or C):

```

// C / C++
int counter = NULL;

```

You will get a message during compilation:

warning: converting to non-pointer type 'int' from NULL [-wconversion-null]

In C#, this is not just a warning, it's an error and the code won't even compile. So, the following C# code doesn't work:

```

// C#
public class Generics<T> {
    public T data;

    public Generics() {
        this.data = null;
    }
}

public class Program {
    public static void Main(string[] args) {
        Generics<int> obj = new();
        System.Console.WriteLine(obj.data);
    }
}

```

We get the message:

Generics.cs(5,17): error CS0403: Cannot convert null to type parameter 'T' because it could be a non-nullable value type.

We need to ensure that the generic type is a nullable type. All classes are nullable types, so we need to ensure T is a class. To do this, we can use generic constraints.

```
// C#
public class Generics<T> where T : class {
    public T data;

    public Generics() {
        this.data = null;
    }
}
```

Now any time this class is instantiated, we must give it a type that is a class. So "int" won't work. We can change it to "string" since this is a class.

```
// C#
public class Program {
    public static void Main(string[] args) {
        Generics<string> obj = new();
        System.Console.WriteLine(obj.data);
    }
}
```

Now it compiles just fine. C# generic type constraints are a powerful feature and is definitely something to research more into.

Java has generic constraints as well.

```
// Java
class Generics<T extends Number> {
    public T data;

    public Generics() {
        this.data = null;
    }
}

public class Main {
    public static void main(String[] args) {
        Generics<Integer> obj = new Generics<>();
        System.out.println(obj.data);
    }
}
```

This ensures that T is a Number (which is Integer, Float, Double, or other numeric reference type).